

Oleksii Odarchuk

Backend Developer (Go) · Creator of rombik and the Piton language



Lviv, Ukraine +380 68 645 61 42 me@ishawyha.dev
@NeShawyha github.com/OlexiyOdarchuk ishawyha.dev

Backend engineer (Go) with a product mindset and half a year of commercial experience. 1st at BEST Hackathon 2026, 3rd at Mate Hackathon, a personal invitation to DOU Day 2026. From hackathon MVPs to production Go services.

1st

place · BEST Hackathon 2026

10 langs

in my own rombik engine

1000×

speed-up (SHMiner)

150+

bot users (first year)

EXPERIENCE

Backend Developer (Go)

October 2025 — present

SkyService

- Design and build **architecturally complex Go microservices from scratch** for a production platform: interface-driven (hexagonal) architecture, inter-service communication, work under real production load.
- Designed an AI orchestrator — a web analog of claude-code**: a control-plane / execution-plane split — Claude runs through a tool-use loop while **all agent code executes inside isolated Docker sandboxes**. Core guarantee, **enforced by an automated test**: secrets (Anthropic key, git tokens) **never reach the sandbox**. MCP-over-SSE, credential-free git (GitHub App), an **egress-allowlist proxy** against exfiltration, a container-lifecycle supervisor.
- AI support-bot platform (RAG)**: semantic memory on PostgreSQL + pgvector (text / vector / hybrid search), a self-learning knowledge graph, an MCP server, schema-per-tenant isolation, token-based billing.

HOW I WORK

- I own products end-to-end** — from the compiler and engine to billing and deploy (rombik, Piton — all mine).
- Security by default** — I find and close vulnerabilities (egress-allowlist, secrets out of the sandbox, ECDSA webhooks).
- I love the foundations, not just CRUD** — compilers, tree-sitter, my own language, distributed systems.
- Fast to ramp up** — a working Go backend in 7 hours at a hackathon; complex production services within half a year.

DOU Day 2026 — personal invitation

Personally invited by the organisers to the conference with a free ticket — while still a student.

TECH STACK

Languages: Go Python C

TypeScript

Backend: Gin Fiber REST

WebSockets Goroutines JWT

OAuth 2.0

Databases: PostgreSQL

pgvector Redis ClickHouse

Frontend: Svelte Wails

Tailwind

AI: Anthropic Claude OpenAI

MCP RAG fal.ai

DevOps: Docker Kubernetes

Linux (Arch) Git TinyGo

Typst

BEYOND CODE

10 years on stage — contemporary and folk dance: performances and productions.

I write for theatre: author of the full-length Christmas féerie **“Uncharted Star”** (seven scenes, original story) — about a girl-engineer who, in a world that turns everyone into obedient cogs, proves that knowledge and a soldering iron beat any magic. I mix engineering thinking with dramaturgy.

EDUCATION & LANGUAGES

Network Technologies

IT College NULP, 2nd year
2024 — present

IT Class

Science Lyceum, Rivne
2022 — 2024

Ukrainian — native · **English** — B1

OPEN SOURCE

I publish libraries, bots and my own language **Piton** — [@OlexiyOdarchuk](#). Commercial rombik is private, but with a public API. Demos and contact — [ishawyha.dev](#).

WINS & HACKATHONS

1st Place — Lead Backend & Sensor Fusion

April 18–19, 2026

BEST Hackathon 2026 — GPS-less Drone Trajectory

- Algorithm to **reconstruct a UAV’s trajectory without GPS** from IMU/barometer logs. Sensor fusion in **Go + C**: Go for the pipeline & integration, C for the hot numerical kernels.
- 1st place among top teams** — judges singled out the architectural resilience: modular split telemetry / integration / simulation let us swap datasets without rewriting.

3rd Place — Backend Engineer March 14, 2026

Mate Hackathon (50 participants, 10 teams)

- Designed and built a Go backend from scratch in **7 hours** for an AI-powered MarTech tool that automates competitor-ad analysis.
- Dual AI pipeline: GPT-4o for trigger analysis + fal.ai for creative generation; backend routed all AI calls under load. Beat teams with 5+ years of experience.

Security Researcher → Job Offer

2026

College Blockchain System “Student Hryvnia”

- Discovered and exploited a critical vulnerability: the **backend never checked transaction amounts** (only the frontend) → arbitrary negative-value transfers. Reported responsibly — received a job offer.
- Reverse-engineered the official JS miner via Webpack bundles — then a faster Go replacement (SHMiner — in Projects).

KEY ENGINEERING DECISIONS

- 1000× speed-up** — rewrote SHA-256 from JS to Go with a goroutine worker-pool: same machine, a thousand times faster (SHMiner).
- Pixel-for-pixel to DSTU** — byte-exact golden tests for flowchart rendering across 10 languages: zero drift from the standard (rombik).
- Secrets out of the sandbox** — an egress-allowlist proxy + an automated test proving keys and tokens never reach the agent’s code (booreviy).
- A compiler from scratch** — parser → IR → layout → render on tree-sitter; craft carried over from my language Piton.

PROJECTS

FLAGSHIP · COMMERCIAL micro-SaaS

[website](#) → [API](#) →

◆ rombik — Flowchart Generator

My first full-fledged commercial micro-SaaS · code in 10 languages → a DSTU 19.701-90 / ISO 5807 flowchart or structogram

Designed and run entirely by me. A **Go engine** — a compiler pipeline (parser → IR → layout → render) on **tree-sitter** for **10 languages** (Python, JavaScript, TypeScript, C, C++, C#, Java, Go, PHP, Pascal); flowcharts and Nassi-Shneiderman structograms, pixel-for-pixel to DSTU 19.701-90 / ISO 5807, byte-exact golden tests. Web editor (pan/zoom, lasso-splitting), 8 formats (SVG/PNG/PDF/Word·Visio·draw.io as native shapes/Typst/Excalidraw). **Billing runs on my own go-monobank-sdk** (HMAC webhooks, grants, receipts). Compiler craft carried over from my language **Piton**. Public **API**, **CLI**, **MCP server**, an AI skill and describe→flowchart via AI. Free watermarked PNG, a clean export needs an account.

Go · Gin

tree-sitter

API · CLI · MCP

go-monobank-sdk

PostgreSQL

DSTU

OPEN SOURCE SDK

go-monobank-sdk — [github](#) → Go SDK for every monobank API

Full coverage of **personal**, **provider** (with monoКЕП), **business** for legal entities, **acquiring** (subscriptions, monipay button, split-receivers, T2P) and **Installment Purchases**. ECDSA webhook verification with auto-rotating keys, drop-in `http.Handler`, typed money/currency/MCC, HMAC-SHA256 for installment callbacks, a fake monobank server for tests, and a separate OpenTelemetry sub-module.

Go ECDSA

HMAC-SHA256

OpenTelemetry iter.Seq2

SDK CONSUMER

monokasa — Telegram [github](#) → Ticket-Selling Bot

Sells tickets to a single show through a monobank jar and issues PDFs with QR codes scanned at the venue entrance. Live consumer of my own **go-monobank-sdk**: webhook auto-registers at startup, reservations get a 15-minute hold, and show-time reminders fire at-most-once even across restarts.

Go SQLite telebot

PDF + QR

go-monobank-sdk

1000× FASTER

SHMiner — Desktop [github](#) → Crypto Miner

Cross-platform desktop miner built with **Wails (Go + Svelte/TypeScript)**. Rewrote SHA-256 hashing from JS to Go with a goroutine worker pool — achieved **1000× faster hashrate** than the official client on identical hardware. AES-encrypted wallet storage, real-time WebSocket dashboard, multi-wallet, zen mode.

Go Svelte TypeScript

Wails WebSockets

Goroutines

150+ USERS

AbitAssistant Bot [github](#) →

Telegram bot helping Ukrainian students track university admission rankings. Reverse-engineered closed third-party APIs to fetch live competition data. **150+ active users in first year**, marketing campaign planned for 2026.

Python Aiogram

PostgreSQL BeautifulSoup

Pandas Docker

LinkShortener Bot [github](#) →

Telegram URL shortener with QR generation and geo analytics. PostgreSQL for storage, Redis for redirect caching, ClickHouse for real-time click analytics (country, city, platform). Docker, GeolIP2.

Go PostgreSQL Redis

ClickHouse Docker

Telegram API

MY OWN LANGUAGE

Piton — My own [github](#) → [docs](#) → programming language

Ukrainian transliterated language + interpreter, from scratch in **Go without yacc/bison**: recursive descent parser, tree-walking evaluator, Turing-complete. Flowcharts via a built-in **D2**-based engine. Interpreter compiled to **WebAssembly via TinyGo** (220 KB).

Go TinyGo WebAssembly

D2 Nix